

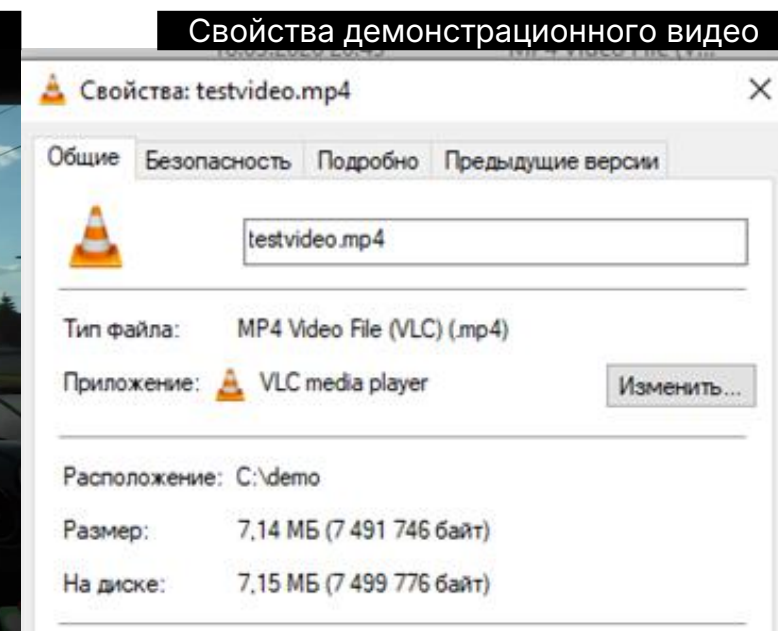
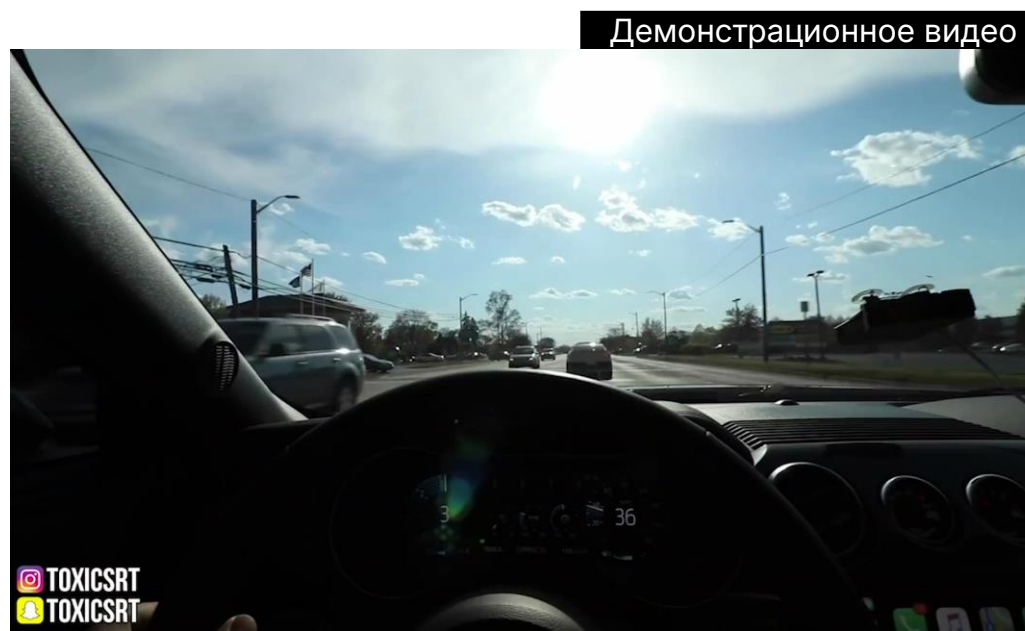
Разработка системы для быстрого декодирования видео в формате AV1

Выполнил студент группы РИС-22-2 Берсенёв Илья

Научный руководитель В. В. Ланин

Кодирование сжимает видео в 300 раз

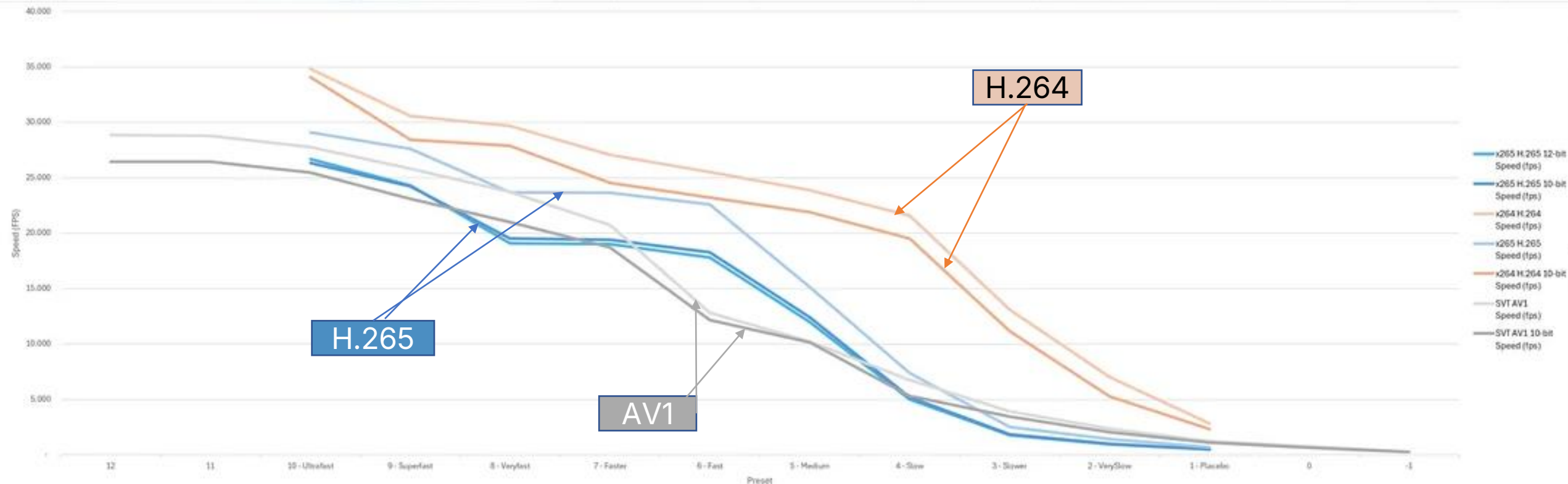
- $1920\text{px} * 1080\text{px} * 30\text{fps} * 3\text{channels} * 1\text{byte} * 14\text{s} = \sim 2491\text{Mb} = \sim \mathbf{2.5Gb}$
- Размер после кодирования с помощью H.264: $\sim \mathbf{7.14Mb}$
- Разница в размерах – более чем в 300 раз
- Высокая эффективность сжатия = высокая нагрузка на CPU



Сравнение скорости кодирования популярных кодеков

- H.264 (2003) -> H.265 (2012) -> AV1 (2018)
- Уменьшение скорости кодирования означает уменьшение скорости декодирования

Зависимость fps от качества выходного видео для различных поколений кодеков



Бизнес-задача Comexr

- TAPe индексатор функционально эквивалентен однонаправленному видеоэнкодеру
- **Проблема:** индексация TAPe в среднем в **24** раза быстрее чем видеodeкодирование
- **Следствие:** узкое место в пайплайне системы
- **Цель:** ускорить декодирование видео чтобы устранить узкое место



Гипотеза, объект и предмет исследования

Объект исследования

процесс восстановления
видеоданных из закодированного
состояния

Предмет исследования

оптимизированная система декодирования
видео в формате AV1

Цель исследования: разработать и применить оптимизации видеodeкодера AV1 для выходного разрешения 64 пикселя, спроектировать архитектуру системы декодирования с учетом максимальной производительности

Гипотеза исследования: Алгоритмические оптимизации видеodeкодера и архитектурные оптимизации системы могут повысить совокупную производительность на более чем 30%

Ограничения исследования

- Выходное разрешение видео: 64 пикселя
- Однопоточный режим выполнения
- Отсутствие оптимизаций, специфичных для архитектуры
 - NVIDIA CUDA
 - SIMD (intel AVX, SSSE, SSE, etc.)
- Сохранение качества видео выше 15db по метрике PSNR

Анализ аналогов

- Не было выявлено решений данной или похожей задачи, так как потеря качества в современных кодеках недопустима
 - Исключение – кодеки для embedded, но в данных исследованиях снижается CPU нагрузка, а не ускоряется кодек
- Было выявлено 3 видеodeкодера: DAV1D, RAV1D, libaom
 - Однопоточный режим, без SIMD, CPU: intel i5-12400f

Сравнение видеodeкодеров AV1

Видеodeкодер	Скорость декодирования получасового видео, в секундах	Язык реализации
libaom	679,788	C
DAV1D	374,877	C
RAV1D	555,763	Rust, C

Анализ предметной области

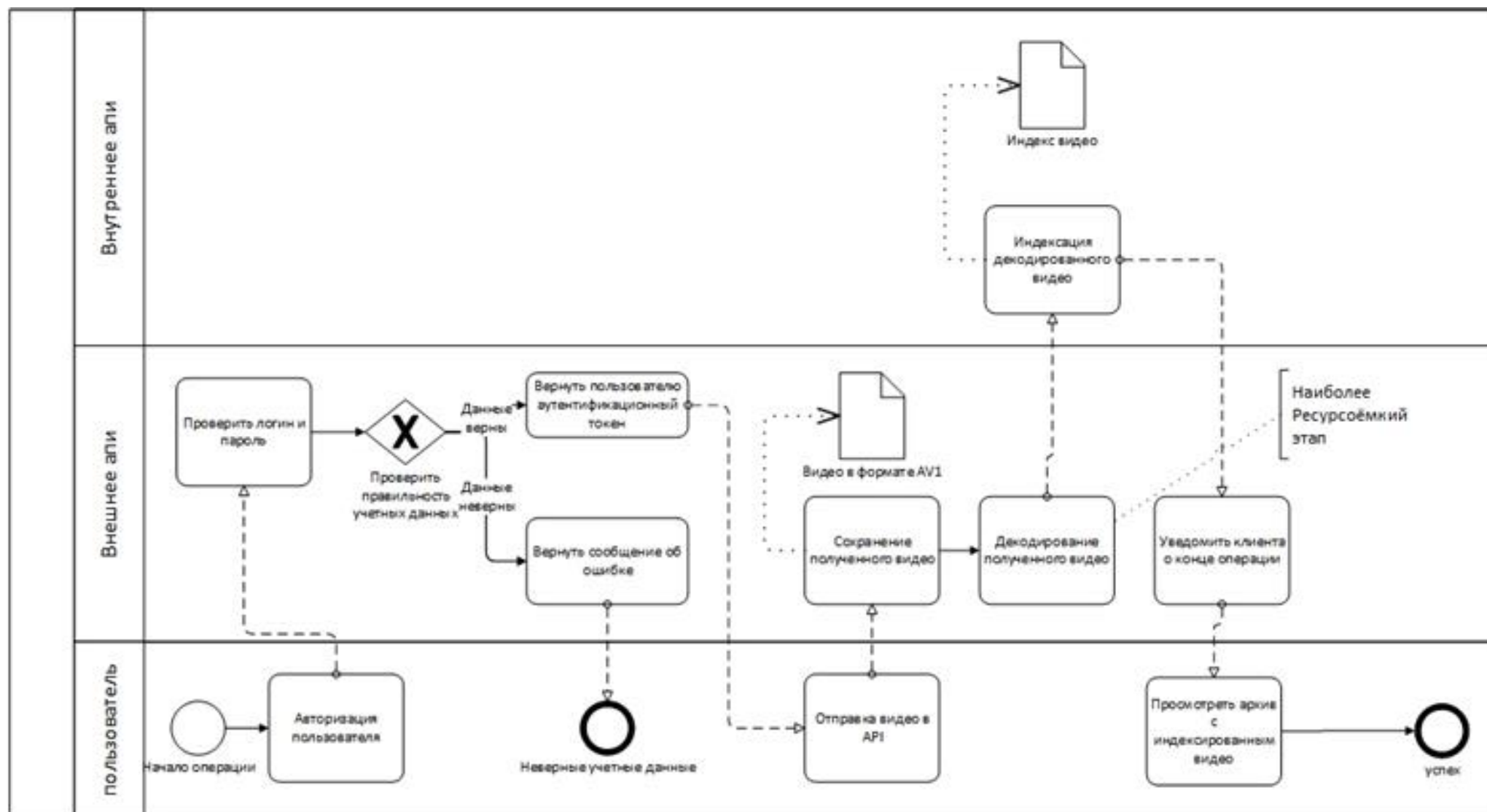
Нефункциональные требования

- Нагрузка при обработке нескольких запросов должна **линейно масштабироваться** при превышении загруженности процессорных мощностей чтобы снизить задержки.
- Декодер AV1 должен быть собран под архитектуру **generic** и не использовать Nvidia CUDA.
- Оптимизированная система должна проходить путь загрузка-декодирование быстрее наивной реализации не менее чем на **30%**.
- Оптимизированный видеodeкодер должен декодировать видео быстрее чем его неоптимизированная реализация не менее чем на **20%**.

Функциональные требования

- Система должна декодировать видео в формате AV1
- Система должна иметь возможность загрузки видео пользователем
- Система должна хранить метаданные о загруженных видеофайлах и статусах задач в реляционной базе данных.
- Система должна уведомлять пользователя о завершении задачи декодирования.

Существующий бизнес процесс



Анализ стандарта AV1

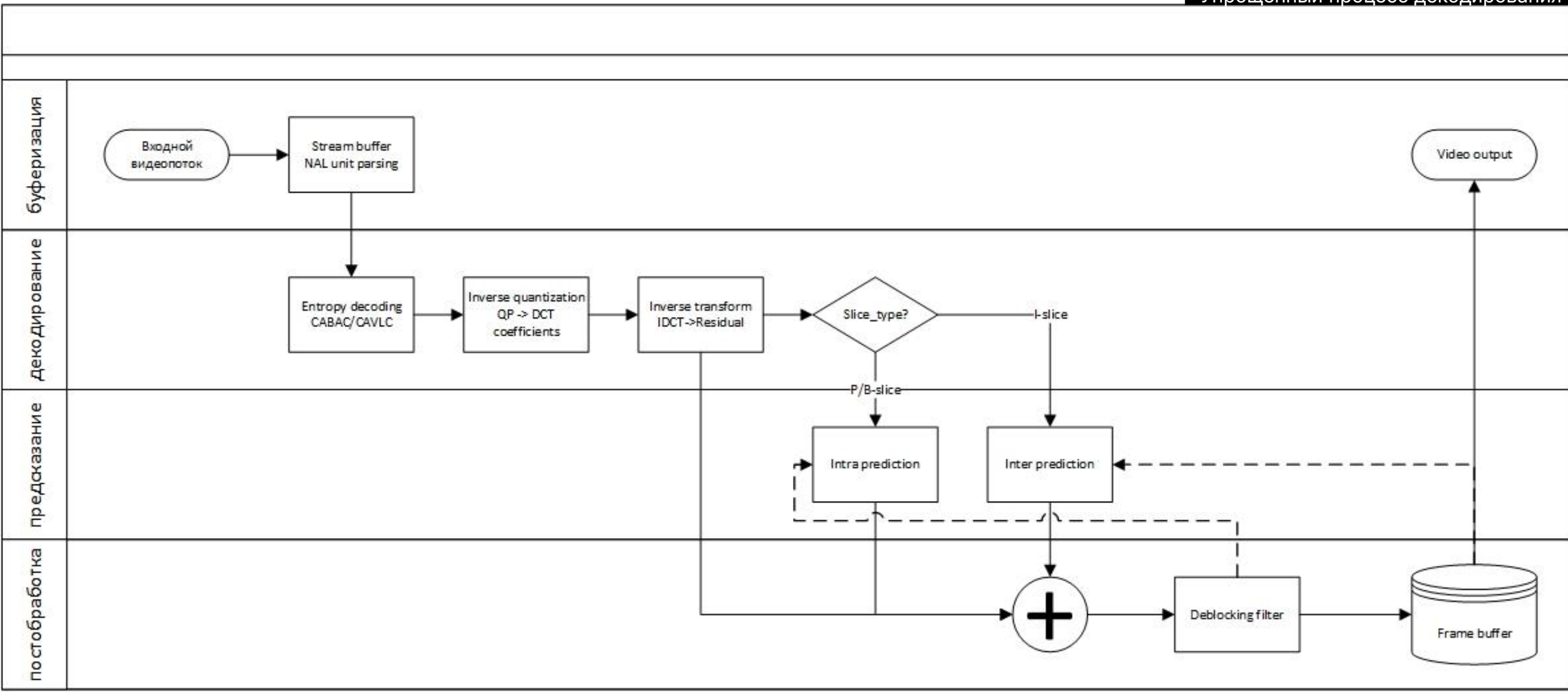
Документы:

- AV1 Bitstream & Decoding Specification v1.0.0-errata
- ITU-T Recommendation H.264



Упрощённый процесс видеodeкодирования на примере H.264

Упрощённый процесс декодирования

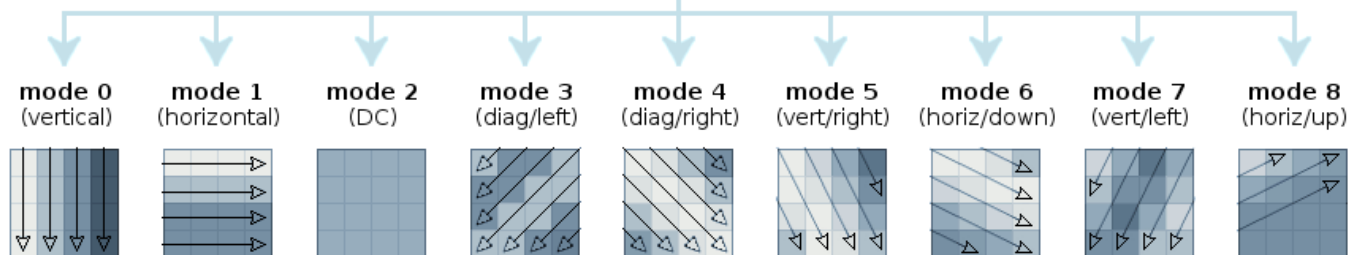


Способы предсказания

- **Intra** – внутрикадровое

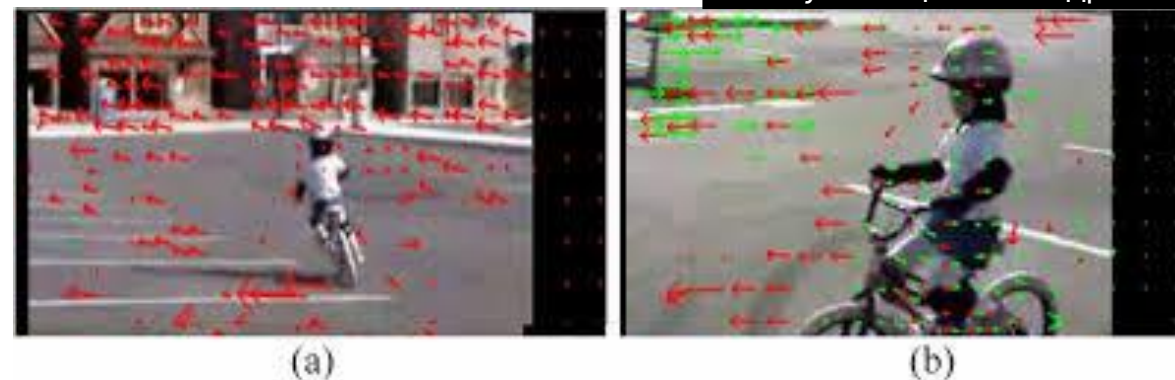
- **Inter** – межкадровое

Визуализация пиксель-по-пикселю предсказания H.264



AVC/H.264 intra prediction modes

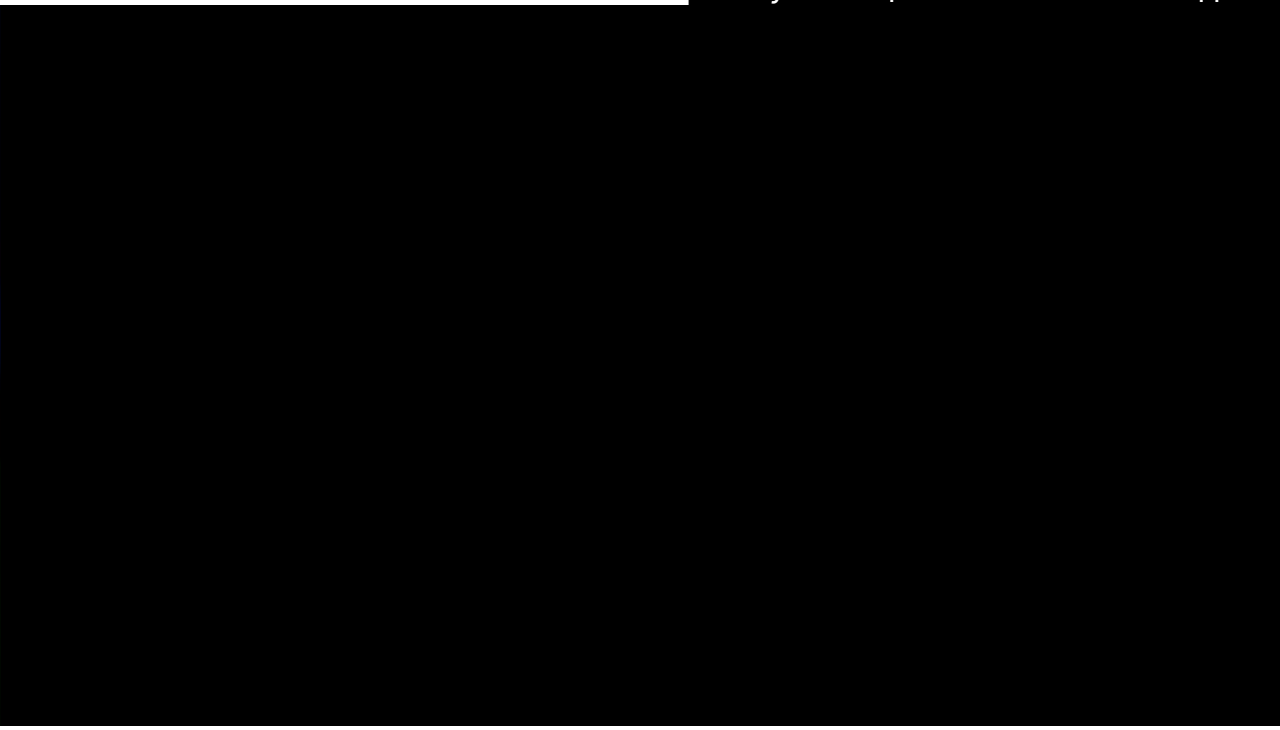
Визуализация MV кадров



Inter: Motion Vectors

- Motion Vectors (MV) – метод кодирования движущихся частей видео с помощью дельты x и дельты y кадра по сравнению с опорным

Визуализация MV стокового видео

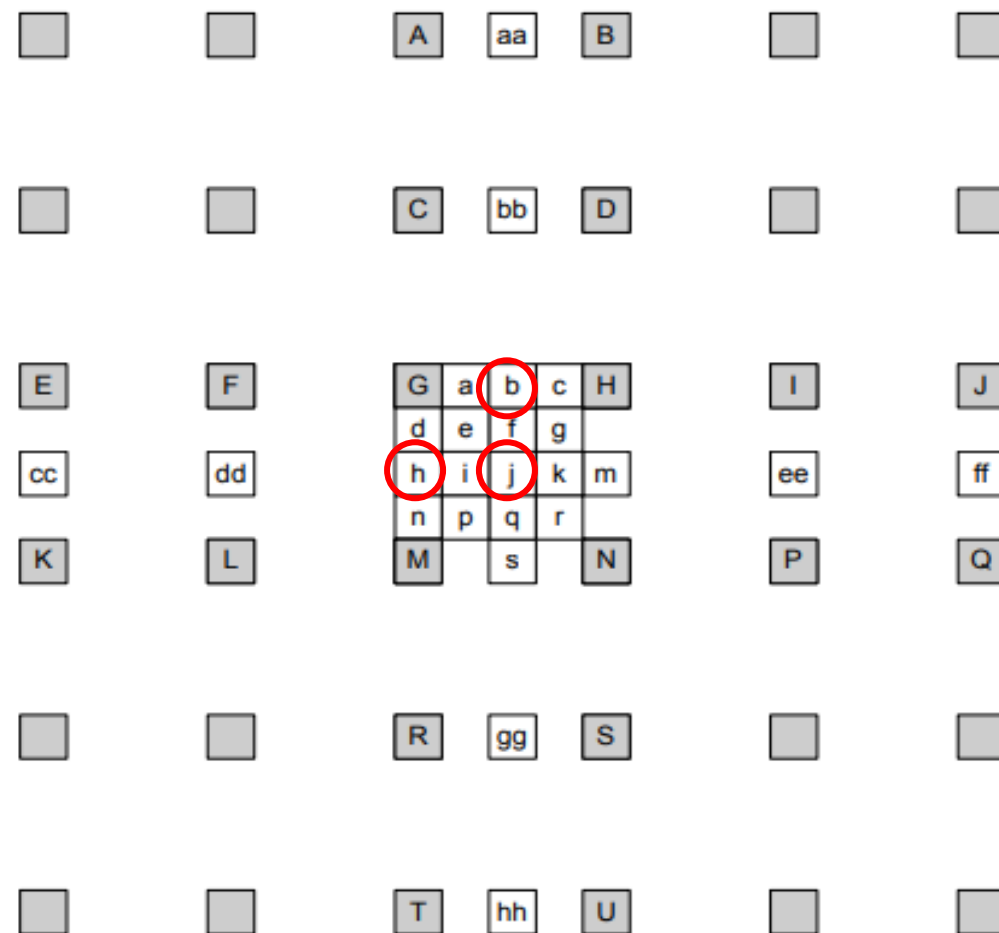


Inter: MV: Интерполяция H.264

• Пример интерполяции для H.264:

- $b = (E - 5 \cdot F + 20 \cdot G + 20 \cdot H - 5 \cdot I + J) / 32 \quad \leftarrow 22 \text{ такта}$
- $h = (A - 5 \cdot C + 20 \cdot G + 20 \cdot M - 5 \cdot R + T) / 32 \quad \leftarrow 22 \text{ такта}$
- $j = ($
 $(cc - 5 \cdot dd + 20 \cdot h +$
 $20 \cdot m - 5 \cdot ee + ff) +$
 $(aa - 5 \cdot bb + 20 \cdot b +$
 $20 \cdot s - 5 \cdot gg + hh)$
 $) / (32 \cdot 2) \quad \leftarrow \text{очень много тактов}$

Схематичное представление интерполяции H.264



Inter: MV: Интерполяция AV1

- 8-tap (AV1) вместо 6-tap (H.264)
- 96 наборов динамических коэффициентов вместо одного статического набора в H.264

Наборы коэффициентов интерполяции AV1

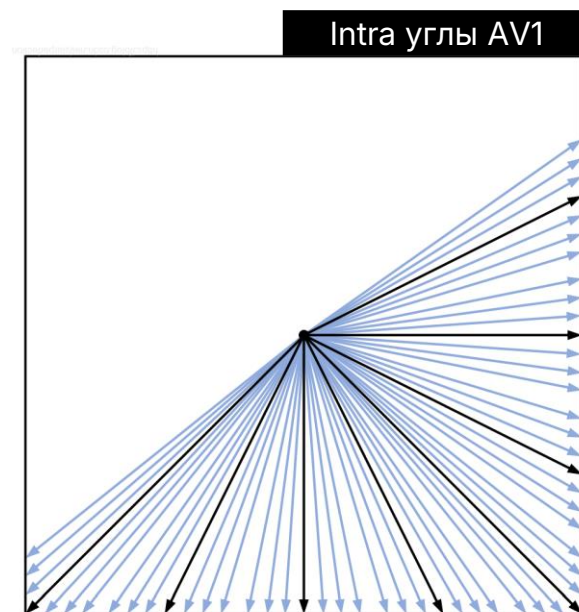
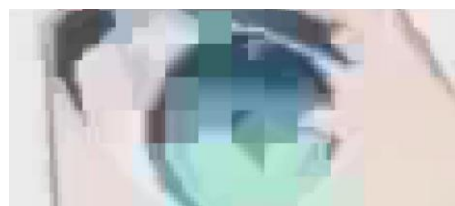
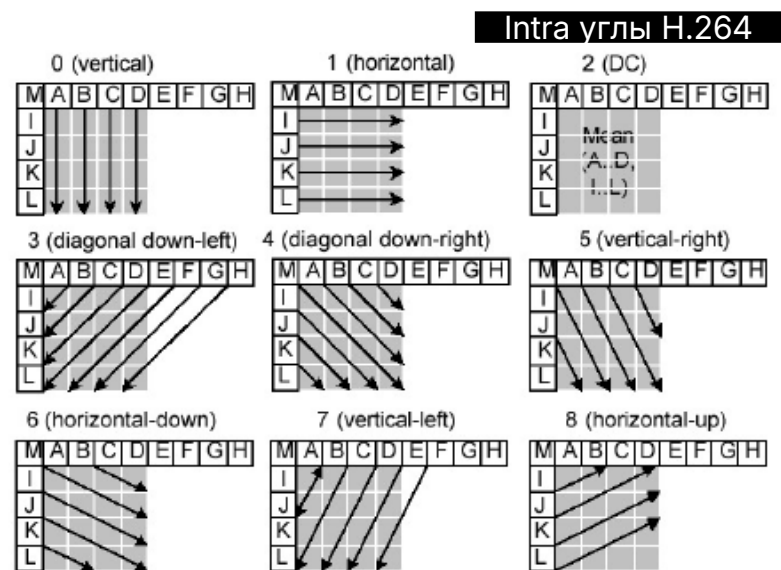
Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-12,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-10,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,-2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,-2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

Intra предсказание

Intra предсказание представляет из себя вычисление пикселя по пикселю.

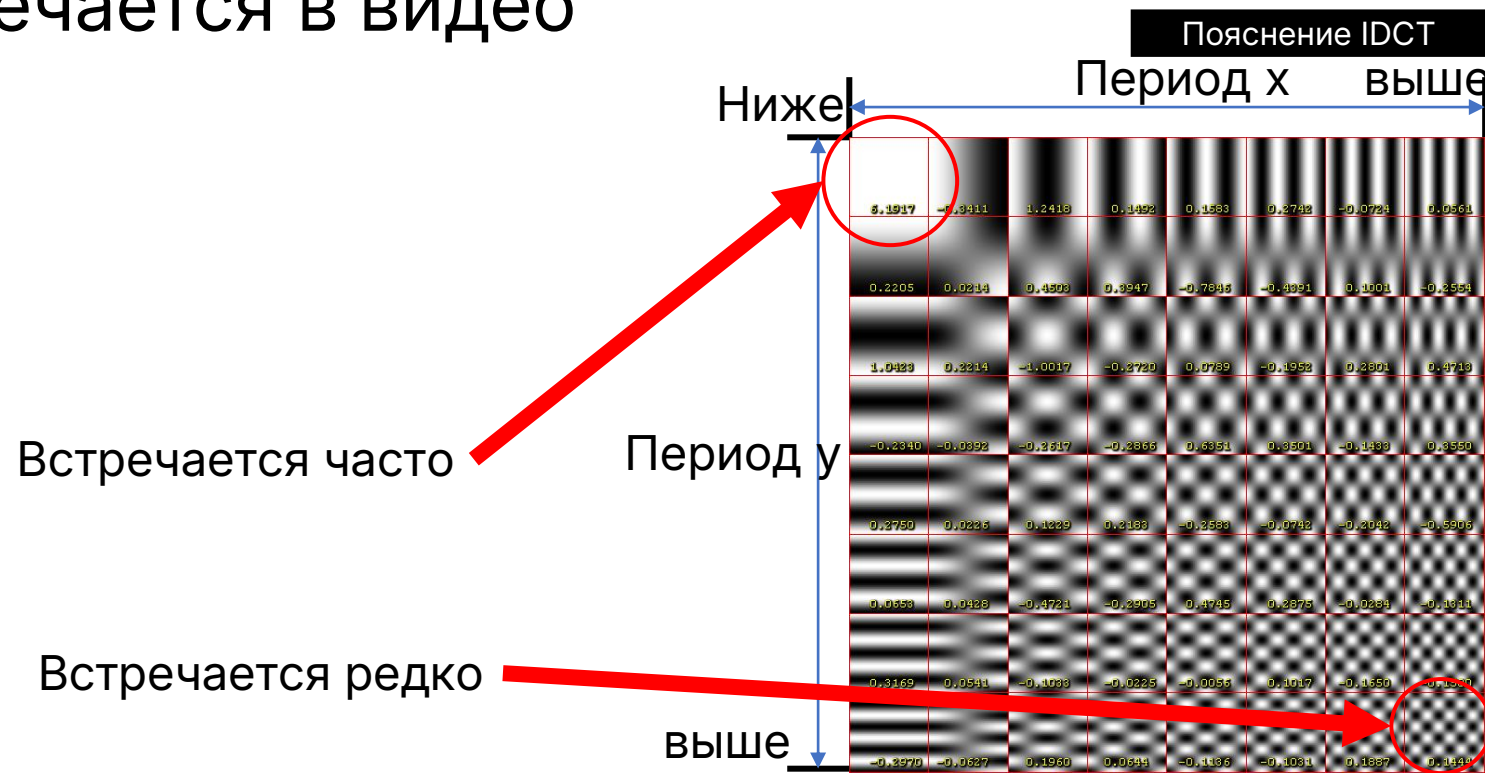
H.264: 9 направлений

AV1: 56 направлений, рекурсивное предсказание



Intra: обратная дискретная косинусная трансформация

Обратная дискретная косинусная трансформация (IDCT) представляет из себя реконструкцию текстуры 16×16 с помощью периодов двух синусоид. Чем больше период, тем реже текстура встречается в видео



Intra: CDEF, DF

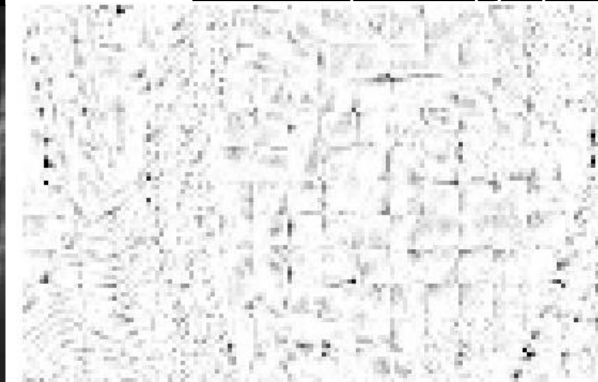
Deblocking filter: фильтр постобработки, устраняющий артефакты блочности после IDCT

Constrained directional enhancement filter: фильтр постобработки, применяющий 8 блочных фильтров направлений к каждому блоку фрейма

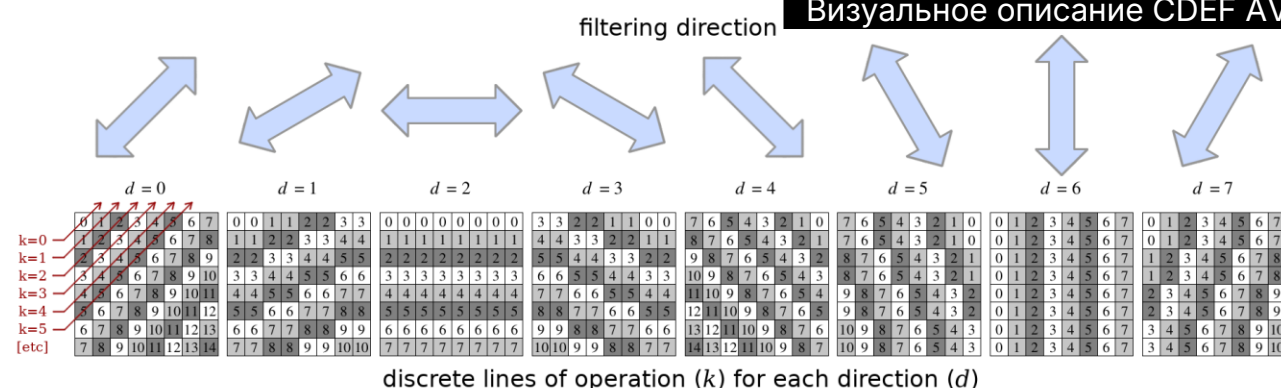
Реконструированное через idct изображение



Ошибки реконструкции



Визуальное описание CDEF AV1



Выводы по анализу

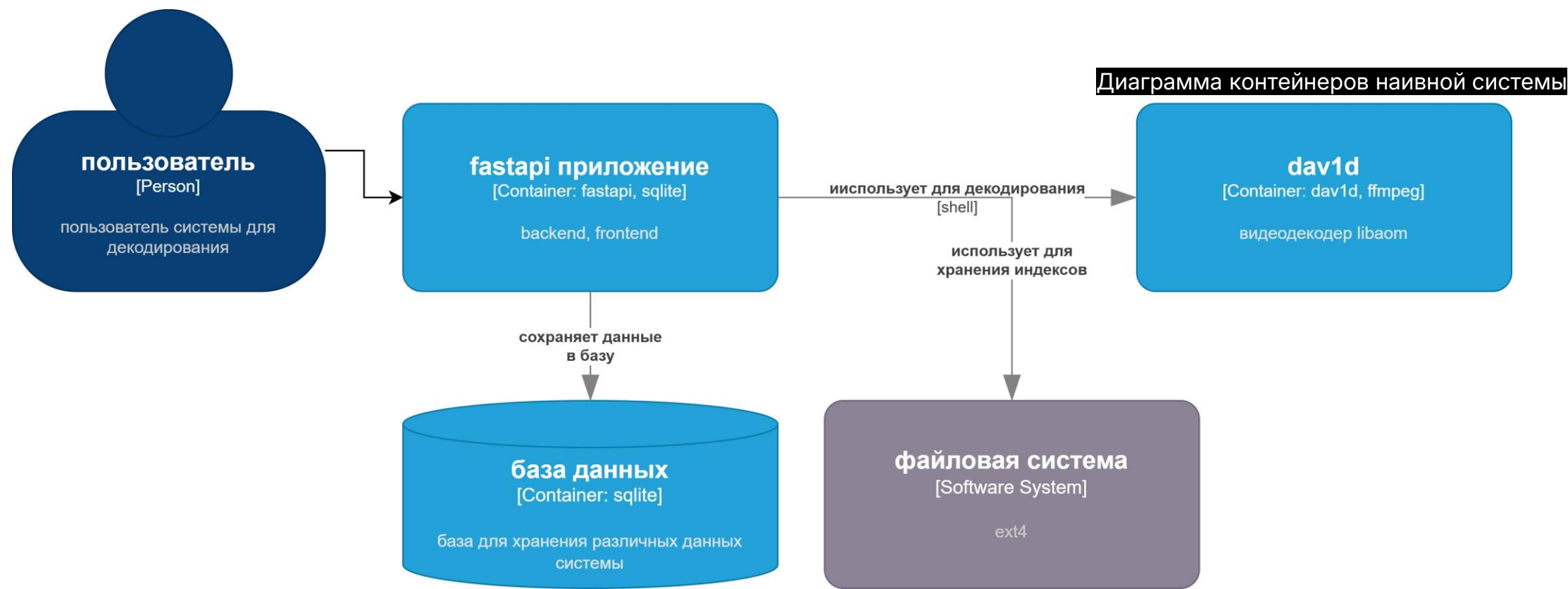
Узкие места:

- IDCT
- DF
- CDEF
- Интерполяция

Проектирование

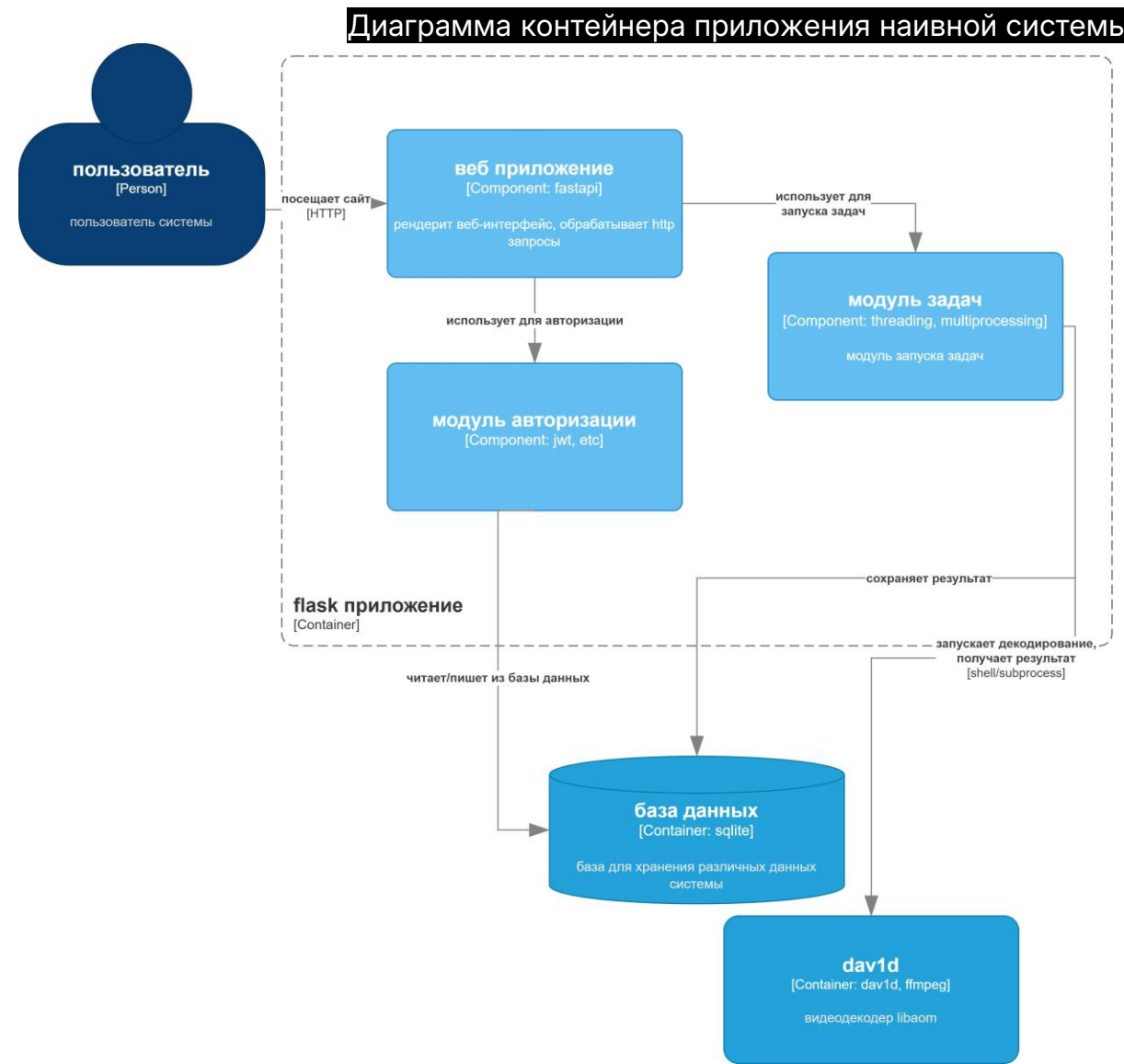
Наивная система

× Не имеет потенциала к масштабированию и оптимизации



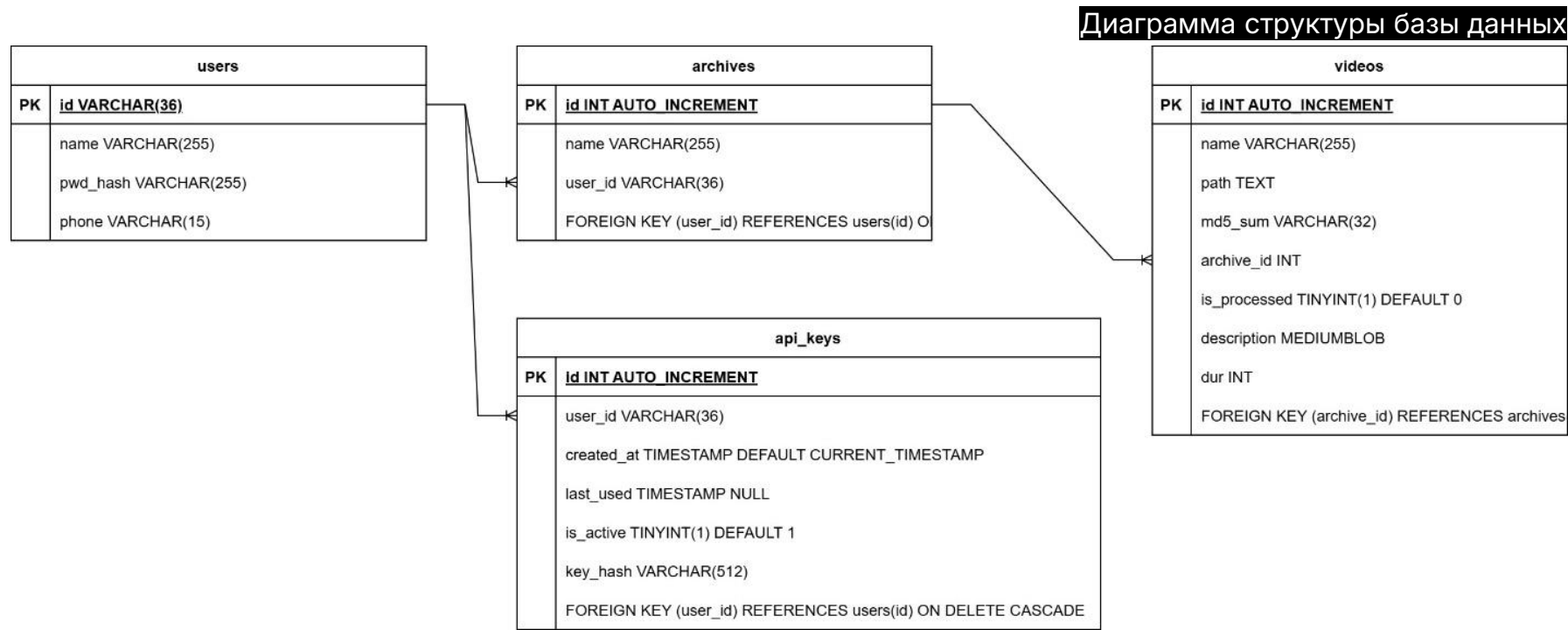
Наивная система: приложение

- Модуль авторизации
 - jwt
- Модуль задач
 - Threading, multiprocessing
- Веб-приложение
 - Fastapi



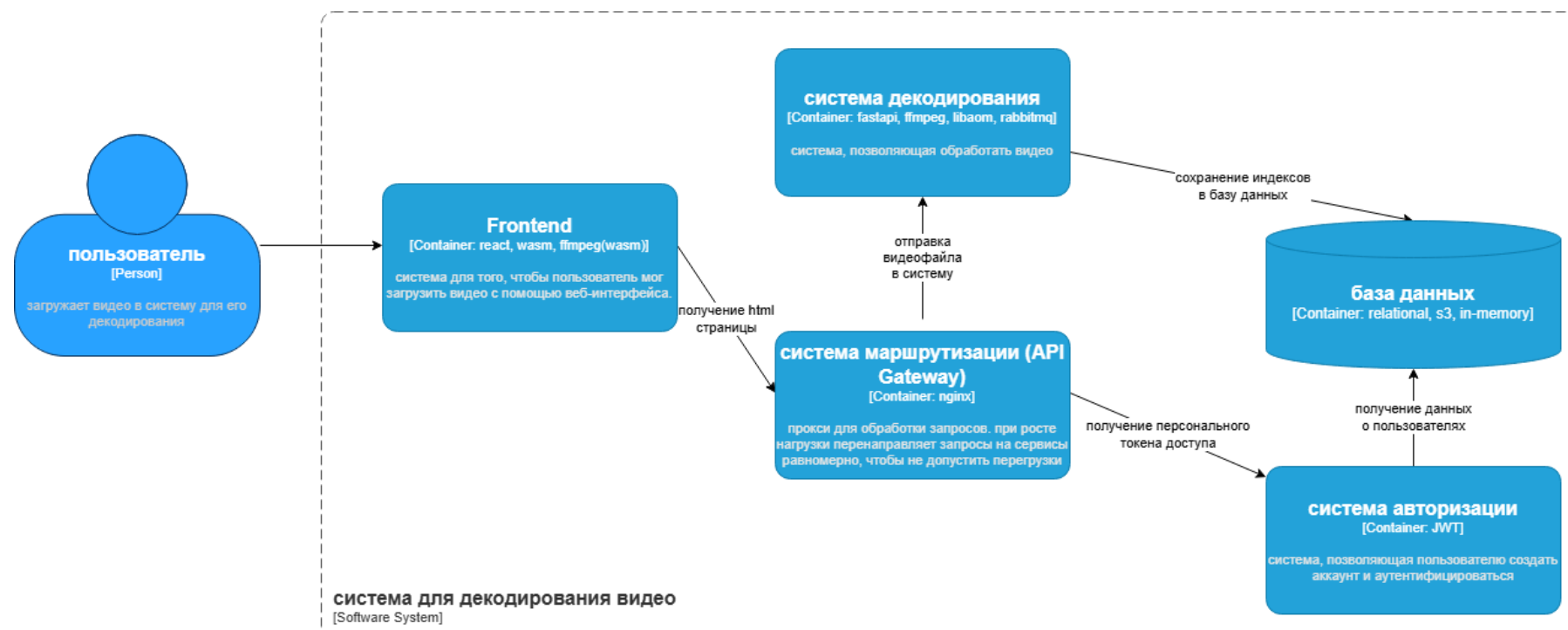
Наивная система: база данных

Структура базы едина для наивной и оптимизированной систем



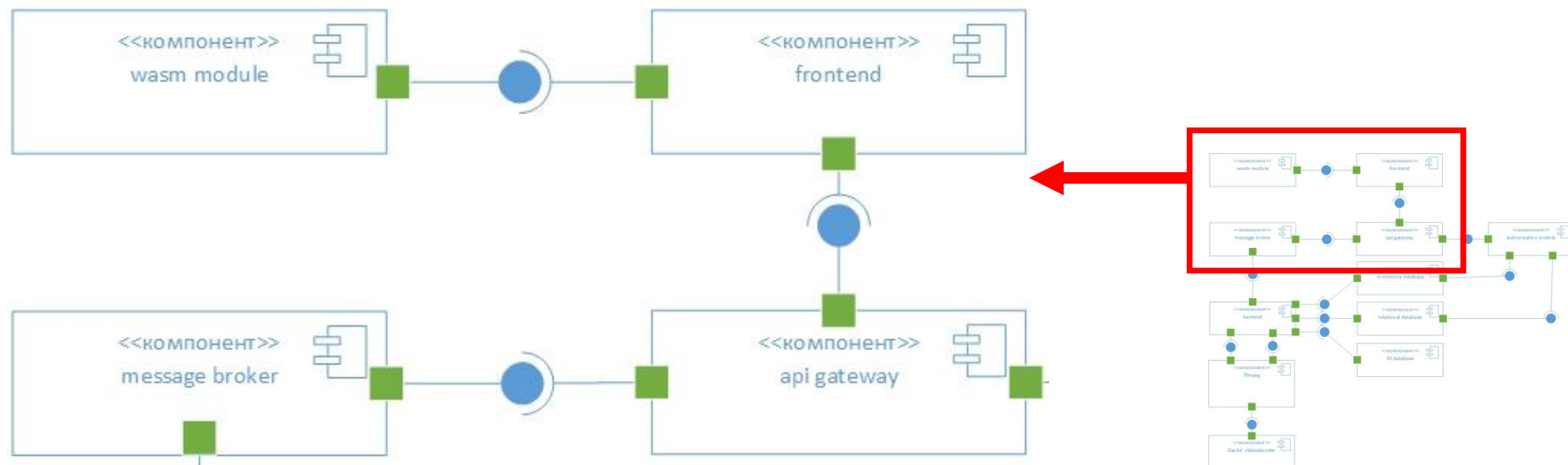
Оптимизированная система

- Frontend
 - Wasm, ffmpeg
- Система декодирования
 - Fastapi, ffmpeg, dav1d
- База данных
 - S3(Ceph), postgresql, Redis
- Система маршрутизации
 - Fastapi, websockets
- Система авторизации
 - JWT



Оптимизированная система: фронтенд

- React
- FFmpeg(WASM)
 - Необходимо для отделения аудио от видео на стороне клиента



Оптимизированная система: фронтенд: дизайн

Дизайн фронтенда состоит из 2 панелей: авторизация и загрузка

Create account

AV1 Video Uploader

USERNAME

PASSWORD

Register

Have an account? Sign in

Dashboard

@root

Logout

Drop AV1 video or click to browse

Load video list

Dashboard

@root

Logout

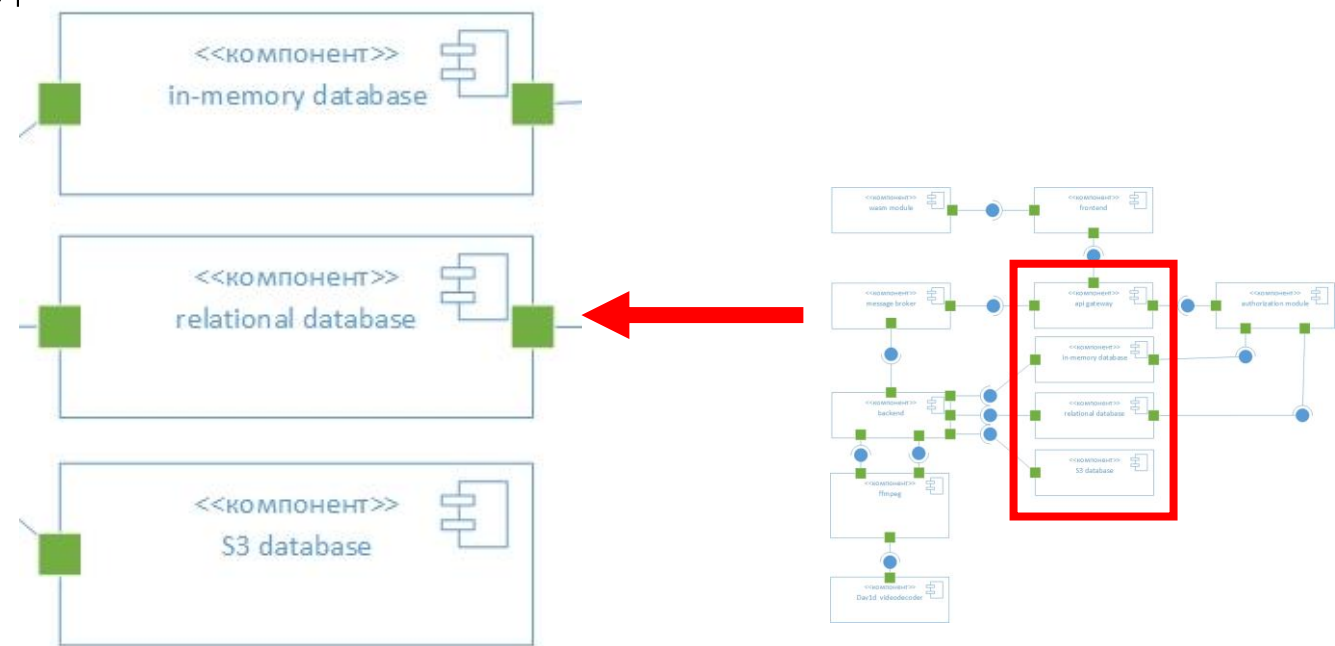
Uploading...

Load video list

Оптимизированная система: базы данных

Базы данных:

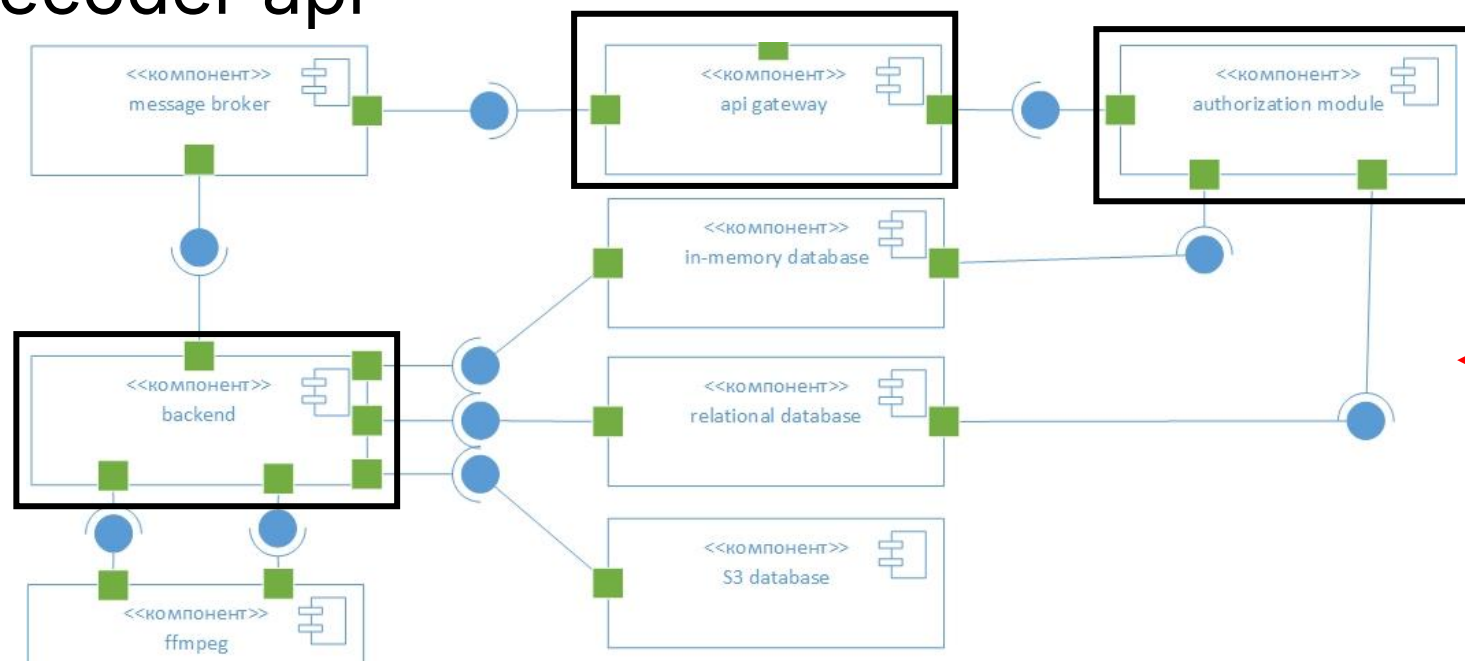
- High latency реляционная база данных – Postgresql
- Low latency нереляционная база данных – Redis
- High latency нереляционная база данных: S3(Ceph)



Оптимизированная система: бэкенд

Компоненты:

- Gateway
- Common api
- Decoder api

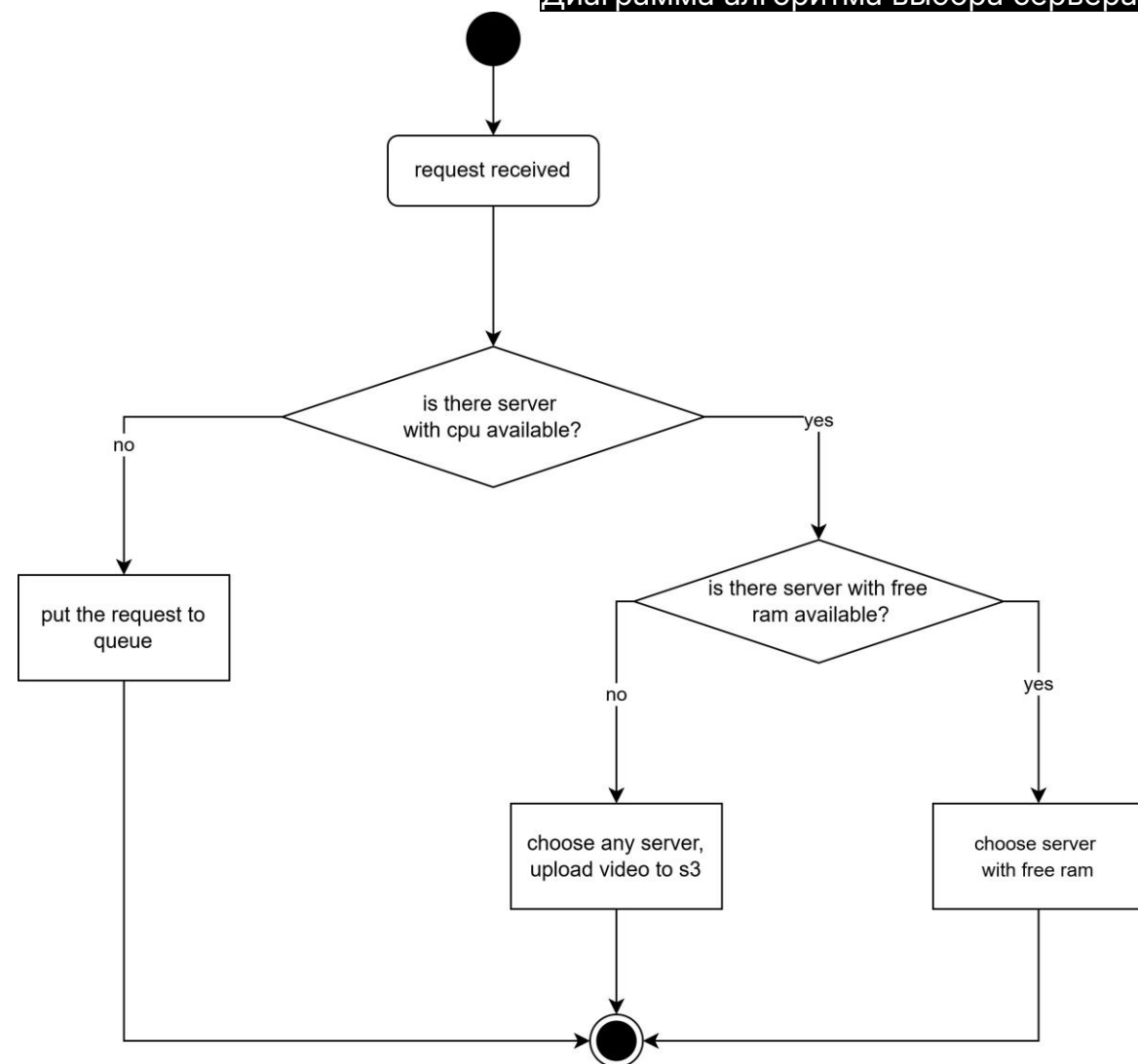


Оптимизированная система: Gateway

Диаграмма алгоритма выбора сервера

Система маршрутизации (Gateway) – прокси и балансировщик.

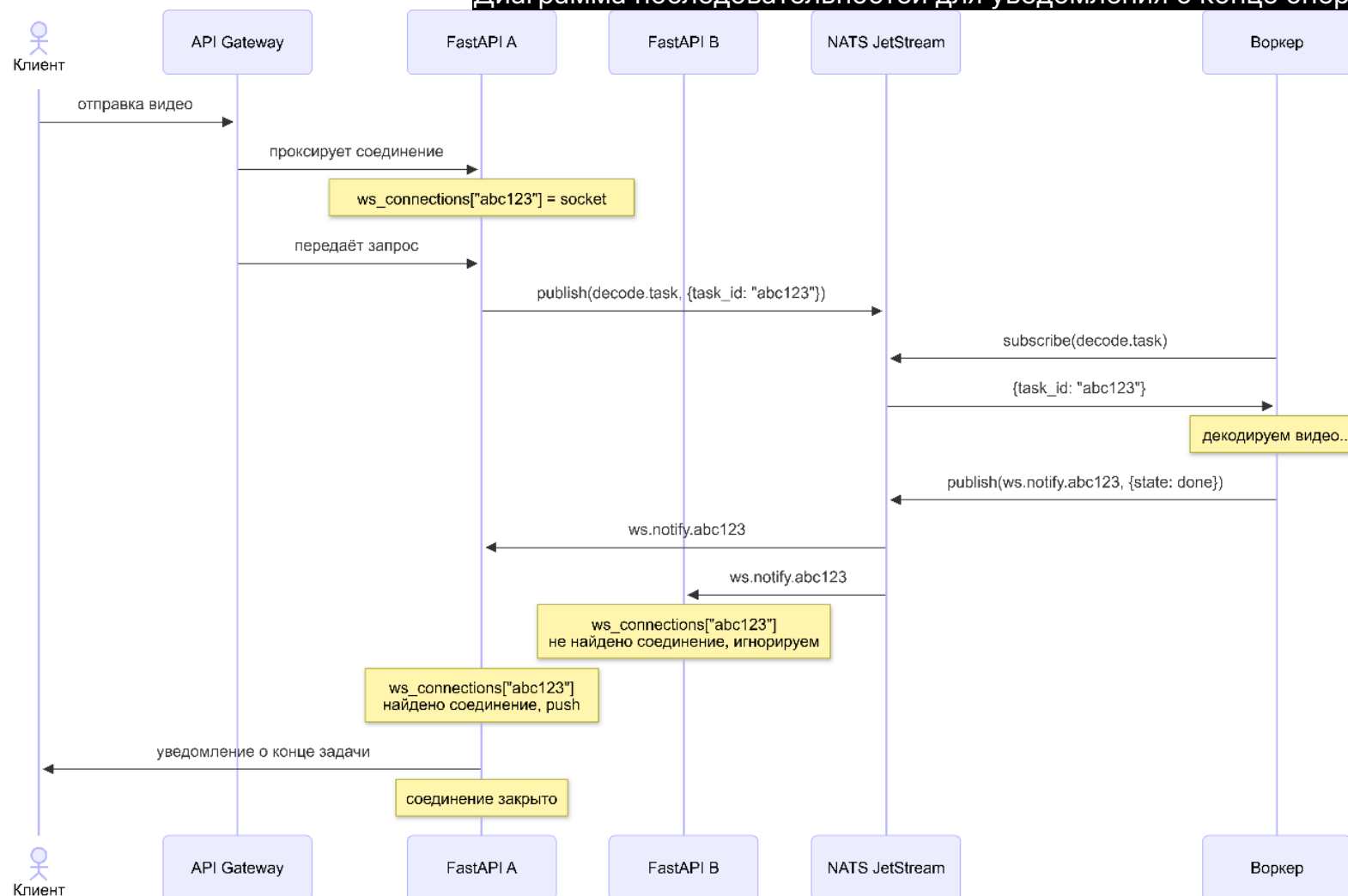
- Собирает информацию от реплик decoder-api через вебсокет
- Проксирует запросы авторизации на common-api
- В зависимости от нагрузки на репликах decoder-api выбирает наименее загруженную реплику



Оптимизированная система: Gateway

Система маршрутизации также выступает держателем всех WebSocket соединений с клиентами, уведомляя пользователя о конце операции

Диаграмма последовательности для уведомления о конце операции

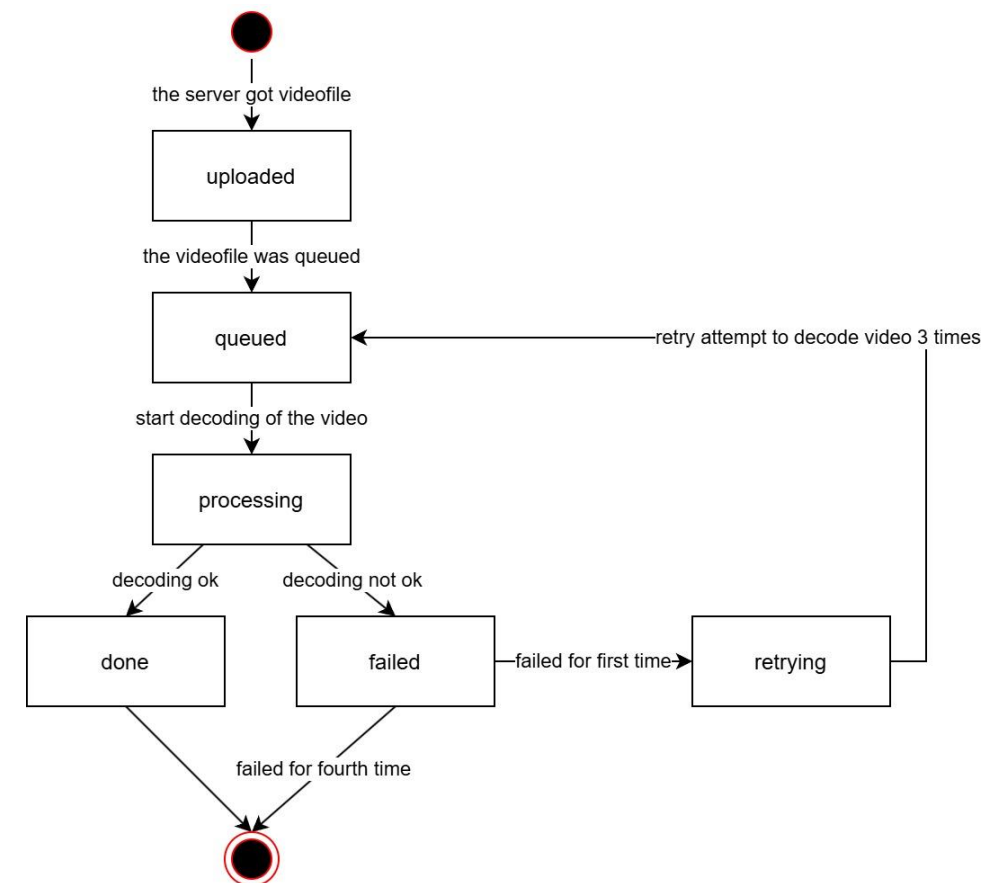


Оптимизированная система: Decoder api

Функции:

- Декодирование видео
 - Из оперативной памяти
 - С диска
- Уведомление пользователя о конце операции (через очередь JetStream)

Диаграмма возможных состояний видео



Реализация: оптимизации DAV1D



DAV1D: метод оценки

Оборудование:

- CPU: AMD EPYC 9554 (1 ядро)
- OS: Ubuntu 22.04.5, без GUI
- DISK: SSD NVMe

Метрики качества:

- PSNR оригинального и декодированного видео
- Покадровая средняя разность luma-компонента

Метрика скорости:

- Скорость от начала декодирования видео до конца декодирования видео
 - 3 итерации, Ubuntu 22.04.5 без GUI

Визуальная репрезентация оптимизации



DAV1D: CDEF, DF

Идея: полностью отключить CDEF, DF, так как на низком разрешении артефакты блочности будут не видны.

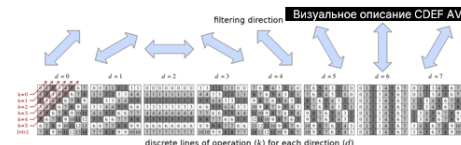
Результат: -26% времени декодирования, артефакты не видны невооруженным глазом.

20 / 63

Intra: CDEF, DF

Deblocking filter: фильтр постобработки, устраняющий артефакты блочности после IDCT

Constrained directional enhancement filter: фильтр постобработки, применяющий 8 блочных фильтров направлений к каждому блоку фрейма



DAV1D: CDEF, LF



DAV1D: 6-тап интерполяция

Идея: сократить количество фильтров интерполяции с 8 до 6. Это можно сделать так как в большинстве случаев первый и восьмой фильтры равны нулю.

Результат: -30.5% времени декодирования, начали копиться небольшие артефакты

Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,0,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-2,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-2,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-2,2,0}	{0,0,2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-2,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-4,2,0}	{0,0,2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-2,0,0}	{0,0,12,52,52,14,0,0}
9/16	{0,2,-12,66,84,-4,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-2,0,0}	{0,0,12,48,54,16,0,0}
10/16	{0,2,-12,58,94,-6,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-4,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,30,0,0}	{0,0,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

DAV1D: 6-таp интерполяция



DAV1D: сокращение набора коэффициентов интерполяции

Идея: увеличить локальность данных, сократив все наборы коэффициентов до 6 и выровняв данные чтобы они гарантированно попали в одну линию процессорного кеша.

Результат: -32.5% времени декодирования, незаметные глазу артефакты

Address	
0x0000	коэффициенты1
0x0020	коэффициенты2 коэффициенты3
0x0040	коэффициенты4,5 коэффициенты6
0x0060	коэффициенты7
0x0080	...
0x00A0	...
0x00C0	...
0x00E0	...
0x0100	...
0x0120	...
0x0140	...
0x0160	...



Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-14,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,58,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-12,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,-2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,-2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

Address	
0x0000	коэффициенты1,2,3,4,5,6
0x0020	
0x0040	

DAV1D: сокращение набора коэффициентов интерполяции



DAV1D: 4-tap интерполяция

Идея: сократить количество фильтров в наборе до 4.

Для этого необходимо сохранить баланс, пример:

1. { -2, 6, -12, 40, 40, -12, 6, -2 }; sum = 64

2. { -12, 40, 40, -12 }; sum = 56

3. { -12, 44, 44, -12 }; sum = 64

Результат оптимизации: -33.1% времени декодирования

DAV1D: 4-тар интерполяция



DAV1D: интерполяция без дробного остатка

Интерполированный пиксель в одной плоскости =

$$(A * X[0] + B * X[1] \dots + (1 \ll 7) \gg 1) \gg 7$$

Идея: упростить вычисление интерполяции, вырезав дробный остаток $(1 \ll 7) \gg 1$.

Результат: -36% времени декодирования

DAV1D: интерполяция без дробного остатка

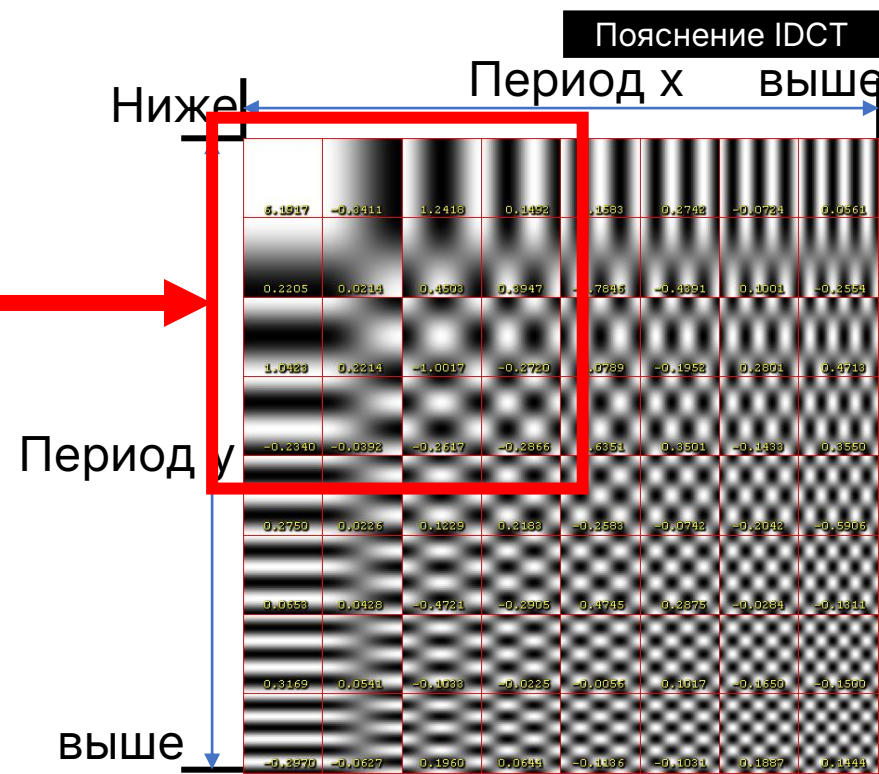


DAV1D: квантизация IDCT

Идея: сократить половину коэффициентов IDCT в каждом измерении и вырезать фильтр винера

Результат: -38.5% от времени декодирования, увеличение блочности в мелких деталях

Частоты которые необходимо оставить



DAV1D: квантизация IDCT



Результаты оптимизации DAV1D

В результате вышеприведённых оптимизаций время декодирования составило 61.5% от оригинальной версии, это означает что скорость декодирования увеличилась на 38.5%, что удовлетворяет поставленной цели.

Прирост производительности в зависимости от оптимизации



DAV1D: тестирование: PSNR

DAV1D: метод оценки

Оборудование:

- CPU: AMD EPYC 9554 (1 ядро)
- OS: Ubuntu 22.04.5, без GUI
- DISK: SSD NVMe

Метрики качества:

- PSNR оригинального и декодированного видео
- Покадровая средняя разность luma-компонента

Метрика скорости:

- Скорость от начала декодирования видео до конца декодирования видео
- 3 итерации, Ubuntu 22.04.5 без GUI



PSNR: вычисление с помощью FFMPEG, numpy

- Минимальное значение: 21.02дБ
- Среднее значение: 26.2дБ
- Максимальное значение: 35.38дБ



PSNR = 40 dB



PSNR = 30 dB



PSNR = 20 dB



PSNR = 10 dB

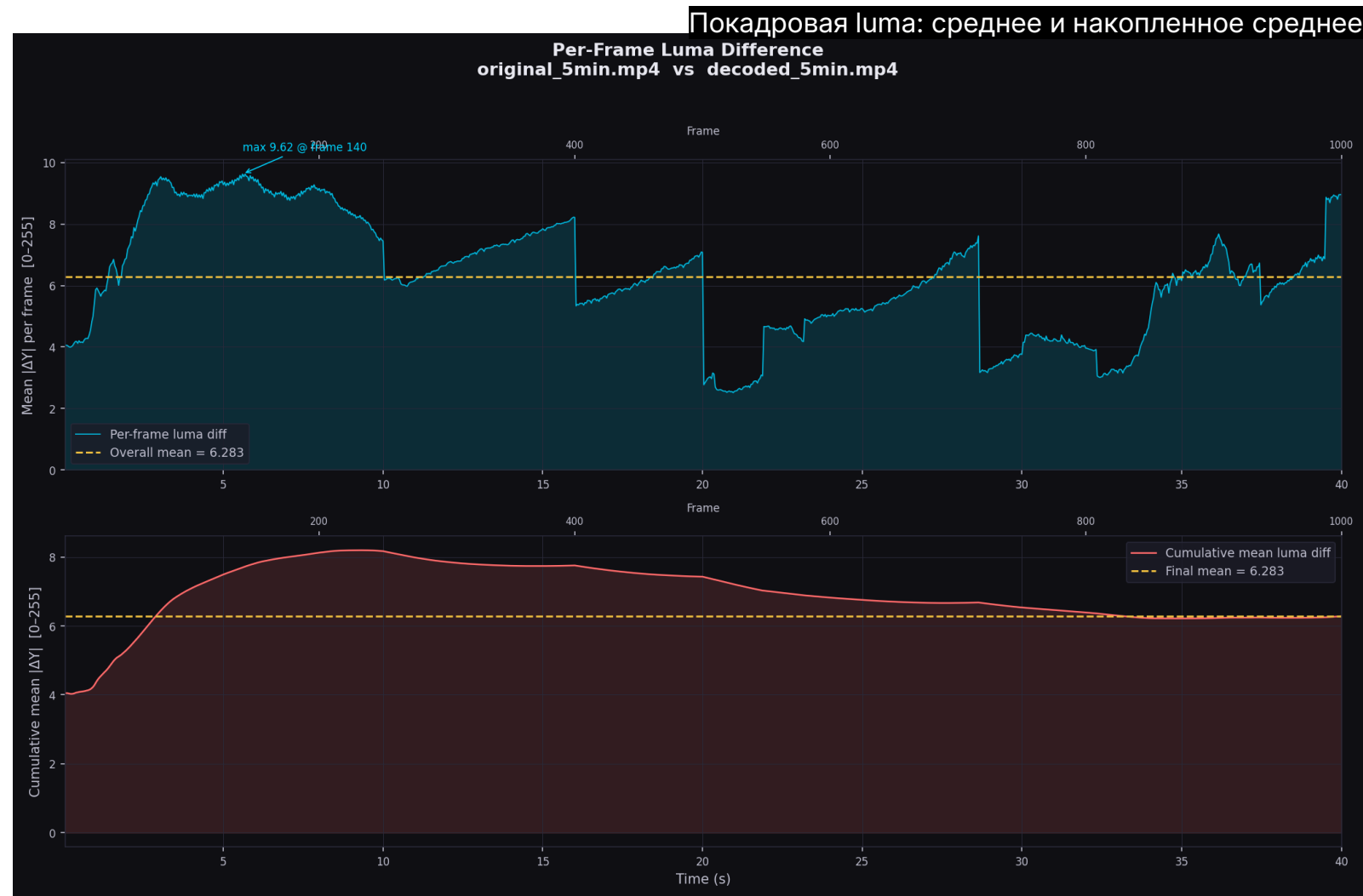


PSNR = 0 dB

DAV1D: тестирование: framewise luma

Покадровая средняя
разница luma: диапазон
от **0** (идеальный
минимум) до **255**
(неприемлемый
максимум)

- Минимальное значение: 2.51
- Среднее значение: 6.28
- Максимальное значение: 9.62



Реализация: оптимизированная система

Оптимизированная система: метод оценки

Оборудование (клиент):

- CPU: Intel i5-12400f,
AMD Ryzen H255
- OS: Linux mint 22.3, Windows 10
- DISK: SSD NVMe

Метрики скорости:

- Общая задержка от начала отправки видео до получения уведомления
 - Последовательная отправка видео через скрипт, минимальное время на 5 итерациях
- Общая пропускная способность системы
 - Количество декодированных видео (Мб) за секунду при параллельной отправке видео на декодирование

Decoder api

Основная функция: Upload

Использует оптимизированный DAV1D

Листинг функции upload

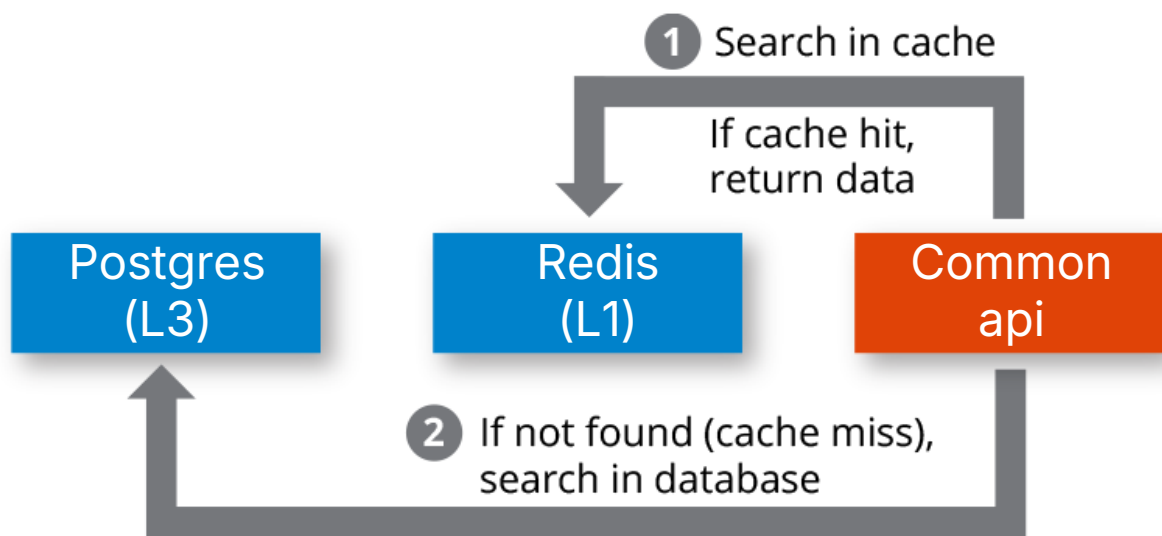
```
25 async def _process_upload(  
26     file: UploadFile,  
27     token: str,  
28     otp_repo: OtpRepository,  
29     video_service: VideoService,  
30     status_repo: StatusRepository,  
31     in_memory: bool = True,  
32 ) -> Response:  
33     payload = await otp_repo.consume_upload_token(token)  
34     await status_repo.create_video('unknown', payload['archive_id'])  
35     file_bytes = await file.read()  
36     frame_bytes = await video_service.decode_first_frame(file_bytes, in_memory=in_memory)  
37     js, _ = await js_connect()  
38     try:  
39         await js.add_stream(name="NOTIFS", subjects=["notifications"])  
40     except Exception:  
41         pass  
42     await js.publish(  
43         "notifications",  
44         payload=json.dumps({"user_id": payload["user_id"], "payload": "123 test"}).encode(),  
45     )  
46     return Response(content=frame_bytes, media_type="image/jpeg")  
47  
48  
49 @router.post("/upload")  
50 async def upload_video(  
51     file: UploadFile = File(...),  
52     token: str = Query(...),  
53     otp_repo: OtpRepository = Depends(get_otp_repo),  
54     video_service: VideoService = Depends(get_video_service),  
55     status_repo: StatusRepository = Depends(get_status_repo)  
56 ):  
57     return await _process_upload(file, token, otp_repo, video_service, status_repo, in_memory=True)  
58  
59  
60 @router.post("/upload_disk")  
61 async def upload_video_disk(  
62     file: UploadFile = File(...),  
63     token: str = Query(...),  
64     otp_repo: OtpRepository = Depends(get_otp_repo),  
65     video_service: VideoService = Depends(get_video_service),  
66     status_repo: StatusRepository = Depends(get_status_repo)  
67 ):  
68     return await _process_upload(file, token, otp_repo, video_service, status_repo, in_memory=False)  
69
```

```
▼ decoder-api  
  ▼ core  
    | config.py  
    | security.py  
  ▼ db  
    | dependencies.py  
  ▼ reporter  
    | __init__.py  
    | reporter.py  
  ▼ repositories  
    | otp_repository.py  
    | status_repository.py  
    | user_repository.py  
  ▼ routers  
    | auth.py  
    | user.py  
    | video.py  
  ▼ services  
    | video_service.py  
  > utils  
  Dockerfile  
  | __init__.py  
  | __main__.py  
  | main.py  
  | requirements.txt
```

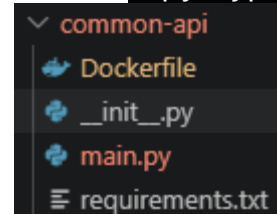
Common api

Апи для авторизации, бизнес функций и прочего.

Оптимизация: cache-miss-like авторизация (redis, postgresql)



структура



Листинг функции авторизации

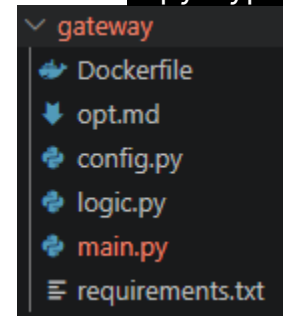
```

81 async def get_current_user(
82     credentials: HTTPAuthorizationCredentials = Depends(security),
83     db: AsyncSession = Depends(get_db),
84 ):
85     token = credentials.credentials
86
87     # 1. Сначала смотрим в Redis (быстрый путь)
88     cached_user_id = await redis_client.get(f"access:{token}")
89     if cached_user_id:
90         # Достаём username из второго ключа
91         username = await redis_client.get(f"user_id:{cached_user_id}:username")
92         if username:
93             return {"id": int(cached_user_id), "username": username}
94
95     # 2. Пропад кэша - идём в БД
96     result = await db.execute(
97         select(User).where(User.access_token == token)
98     )
99     user = result.scalar_one_or_none()
100     if not user:
101         raise HTTPException(status_code=401, detail="Invalid or expired token")
102
103     # Прогреваем кэш для следующих запросов
104     await redis_client.setex(f"access:{token}", ACCESS_TOKEN_TTL, user.id)
105     await redis_client.setex(f"user_id:{user.id}:username", ACCESS_TOKEN_TTL, user.name)
106
107     return {"id": user.id, "username": user.name}
108
  
```

Gateway

- Отказ от прямого проксирования Upload
 - Вместо этого: GET чтобы получить прямую ссылку с одноразовым токеном
- Добавление пороговых значений для загрузки CPU, RAM, DISK MEM: 90%
- Получение информации о загрузке через /ws/replica
- Уведомление клиентов о конце операции через пул вебсокетов на /ws/notify

структура



Frontend

Представляет из себя 2 модуля: FFMPEG WASM и REACT

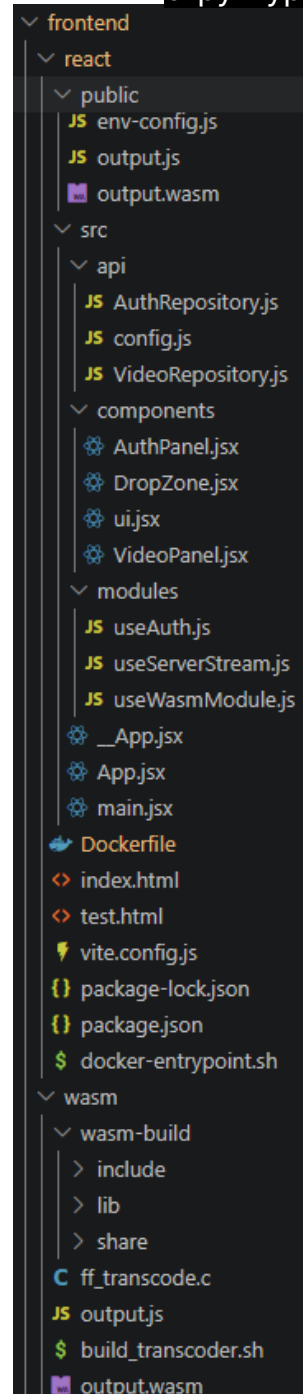
Для компиляции в WASM через Emscripten:

- Были вручную вырезаны функции для обратных трансформаций
- FFMPEG был скомпилирован с флагом `-Oz` для максимальной оптимизации размера
- Было отключено всё кроме муксера и демуксера AV1, а также необходимых протоколов

Результат оптимизации: суммарный размер WASM FFMPEG упал с 50мб до 2мб.

Итоговая конфигурация:

```
emconfigure ./configure --extra-cflags="-Oz -ffunction-sections -fdata-sections -fPIC" --extra-ldflags="-Wl,--gc-
sections -Wl,--strip-all, --strip-debug --closure 1" --prefix=$(pwd)/wasm-build --disable-programs --disable-
shared --disable-doc --enable-cross-compile --disable-sdl2 --disable-iconv --disable-everything --
enable-protocols --enable-demuxers --enable-muxer=obu --enable-parser=av1 --enable-static --enable-
hardcoded-tables --enable-avcodec --enable-avutil --enable-avformat --cc=emcc --target-os=none --
disable-x86asm --disable-inline-asm --arch=x86_64 --disable-d3d11va --disable-d3d12va --disable-dxva2 --
disable-cuda-llvm --disable-audiotoolbox --disable-amf --disable-nvdec --disable-nvenc --disable-large-tests
--disable-asm --disable-xlib --enable-small --disable-debug --disable-dwt --disable-hwaccels --disable-
txtpages --disable-network --disable-pthreads --disable-os2threads --disable-w32threads --disable-
optimizations --enable-pic
```



Развертывание

Весь проект имеет 2 варианта развертывания: через Docker swarm и через Docker compose. При первом варианте реплики декодера самостоятельно устанавливаются на сервера, по одной на сервер включая узел-менеджер.

На данный момент проект развернут с помощью Docker swarm на двух серверах.

52 / 63

Оптимизированная система: метод оценки

Оборудование (клиент):

• CPU: Intel i5-12400f,
AMD Ryzen H255

• OS: Linux mint 22.3, Windows 10
• DISK: SSD NVMe

Метрики скорости:

• Общая задержка от начала отправки видео до получения уведомления

• Последовательная отправка видео через скрипт, минимальное время на 5 итерациях

• Общая пропускная способность системы

• Количество декодированных видео (Мб) за секунду при параллельной отправке видео на декодирование

Тестирование оптимизированной системы

Метрики скорости наивной и оптимизированной систем		
Метрика	Наивная система	Оптимизированная система
Общая задержка от начала отправки видео на сервер до полного декодирования, в секундах, 5 видео	283,4с	165,9с
Общая пропускная способность системы, в МБ/с	1,98Мб/с	4,28Мб/с

Результаты проекта

Общие метрики наивной и оптимизированной систем

Метрика тестирования		Мера	Целевая мера
Скорость декодирования видео (DAV1D)		+38,5%	+20%
Общая задержка от начала отправки видео на сервер до полного декодирования, в секундах		-41,4%	-30%
Общая пропускная способность системы, в МБ/с		+116,2%	-
Потеря качества при декодировании, PSNR	Минимум	21,02 дБ	>15дБ
	Среднее	26,2 дБ	
	Максимум	35,38 дБ	
Покадровая метрика потери качества, 0-255	Минимум	2,51	-
	Среднее	6,28	
	Максимум	9,62	

Код проекта

Оптимизированный
декодер

<https://github.com/ressiwage/DAV2D>



WASM FFMPEG

<https://github.com/ressiwage/FFFMPEG>



Код проекта

<https://github.com/ressiwage/UFAV1VDS>



Источники 1/2

1. Alliance for Open Media. AV1 Bitstream & Decoding Process Specification / Version 1.0.0 with Errata 1 [Электронный ресурс]: технический стандарт / консорциум Alliance for Open Media. – 2021. – URL: <https://aomediacodec.github.io/av1-spec/av1-spec.pdf> (дата обращения: 19.12.2025).
2. Международный союз электросвязи. Сектор стандартизации (ITU-T). *Рекомендация ITU-T H.264 (V15). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services* [Электронный ресурс]. — изд. от 13.08.2024. — (Серия H: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).
3. [Chen Y. An Overview of Core Coding Tools in the AV1 Video Codec / Yue Chen, D. Murherjee, J. Han // IEEE 2018 Picture Coding Symposium \(PCS\). — P. 41-45 – 2018. — DOI: 10.1109/PCS.2018.8456249.](#)
4. Petreski D. Next Generation Video Compression Standards – Performance Overview / D. Petreski, T. Kartalov // IEEE International Conference on Systems, Signals and Image Processing (IWSSIP) — 2023. — P. 1–5. — DOI: 10.1109/IWSSIP58668.2023.10180261.
5. Kolodziejcki W. Speeding Up the AV1 Global Warped Motion Compensation / W. Kolodziejcki, R. Domanski, L. Agostini // 2024 IEEE 15th Latin America Symposium on Circuits and Systems (LASCAS). — 2024. — P. 1-5. — DOI: 10.1109/LASCAS60203.2024.10506160.
6. Yi Y. High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec / Y. Yi, B. C. Song // IEEE Signal Processing Letters — vol. 15 — 2008. — P. 891–894. — DOI: 10.1109/LSP.2008.2001982.
7. [NATS JetStream vs RabbitMQ vs Apache Kafka on VPS in 2025: Throughput/Latency Benchmarks, Exactly-Once vs At-Least-Once, Durability/Replication, TLS/mTLS, and the Best Message Broker for Your Stack / блог компании Onidel \[Электронный ресурс\]. 8.11.2025. URL: <https://onidel.com/blog/nats-jetstream-rabbitmq-kafka-2025-benchmarks> \(дата обращения: 13.02.2026\).](#)
8. [Angular vs. React vs. Vue.js in 2025 / блог компании DLT Software \[Электронный ресурс\]. 20.11.2025. URL: <https://medium.com/@dlsoft/angular-vs-react-vs-vue-js-in-2025-f24b56ff0064> \(дата обращения: 13.02.2026\).](#)
9. [H.264 is Magic / Sid Bala \[Электронный ресурс\]. 2.11.2016. URL: <https://sidbala.com/h-264-is-magic/> \(дата обращения: 13.02.2026\).](#)
10. [Lin Y., Efficient AV1 Video Coding Using a Multi-layer Framework / Wei-Ting Lin, Zoe Liu, Debargha Mukherjee // IEEE 2018 Data Compression Conference. — P. 368-369 – 2018. — DOI: 10.1109/DCC.2018.00045.](#)
11. [SVT-AV1 documentation: ALTREF and Overlay Pictures \[Электронный ресурс\]. URL: <https://gitlab.com/AOMediaCodec/SVT-AV1/-/blob/master/Docs/Appendix-Alt-Refs.md> \(дата обращения: 13.02.2026\).](#)
12. [Salonen N. A Comparative Study of Kafka and NATS : \[дипломная работа\] / N. Salonen. — Turku : University of Turku, 2025. — 62 с. — URL: \[https://www.utupub.fi/bitstream/handle/10024/182402/Salonen_Nino_opinnayte.pdf\]\(https://www.utupub.fi/bitstream/handle/10024/182402/Salonen_Nino_opinnayte.pdf\) \(дата обращения: 14.03.2026\).](#)

Источники 2/2

13. [Comparing NATS, NATS JetStream, and Kafka Performance for Varying Payload Sizes / Nathan B. Crocker](https://medium.com/@nathanbcrocker/comparing-nats-nats-jetstream-and-kafka-performance-for-varying-payload-sizes-3538c94ac56c) [Электронный ресурс]. 24.09.2024. URL: <https://medium.com/@nathanbcrocker/comparing-nats-nats-jetstream-and-kafka-performance-for-varying-payload-sizes-3538c94ac56c> (дата обращения: 14.03.2026).

14. [Comparison of NATS, RabbitMQ, NSQ, and Kafka / блог компании Gcore](https://gcore.com/learning/nats-rabbitmq-nsq-kafka-comparison) [Электронный ресурс]. 20.06.2024. URL: <https://gcore.com/learning/nats-rabbitmq-nsq-kafka-comparison> (дата обращения: 14.03.2026).

15. [Python Flask vs FastAPI vs Django: Framework Comparison 2026 / блог](https://dasroot.net/posts/2026/02/python-flask-fastapi-django-framework-comparison-2026/) [Электронный ресурс]. 07.02.2026. URL: <https://dasroot.net/posts/2026/02/python-flask-fastapi-django-framework-comparison-2026/> (дата обращения: 14.03.2026).

16. [Using H.264 video compression in IP video surveillance systems / блог](https://www.networkwebcams.co.uk/blog/h264-video-compression-in-ip-video-surveillance-systems/) [Электронный ресурс]. 03.04.2009. URL: <https://www.networkwebcams.co.uk/blog/h264-video-compression-in-ip-video-surveillance-systems/> (дата обращения: 23.03.2026).

17. [Международный союз электросвязи. Сектор стандартизации \(ITU-T\). *Рекомендация ITU-T H.264 \(V15\). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services*](https://www.itu.int/rec/T-REC-H.264) [Электронный ресурс]. — изд. от 13.08.2024. — (Серия Н: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — р. 126. — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).

18. [Международный союз электросвязи. Сектор стандартизации \(ITU-T\). *Рекомендация ITU-T H.264 \(V15\). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services*](https://www.itu.int/rec/T-REC-H.264) [Электронный ресурс]. — изд. от 13.08.2024. — (Серия Н: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — р. 101. — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).

19. [Prediction of Transformed \(DCT\) Video Coding Residual for Video Compression / статья](https://www.researchgate.net/publication/261700797_Prediction_of_Transformed_DCT_Video_Coding_Residual_for_Video_Compression) [Электронный ресурс]. 04.2014. — р. 6 — URL: https://www.researchgate.net/publication/261700797_Prediction_of_Transformed_DCT_Video_Coding_Residual_for_Video_Compression (дата обращения: 23.03.2026).

20. [Ren J. Algorithmic Optimization for H.264 Deblocking Filter on Portable Devices / J. Ren, N. Kehtarnavaz // 2007 IEEE International Symposium on Consumer Electronics. — p. 1-6 – 2007. — DOI: 10.1109/ISCE.2007.4382168](#)

21. [Han J. A Technical Overview of AV1 / J. Han, B. Li, D. Mukherjee // Proceedings of the IEEE. — vol. 109, no. 9, p. 1435-1462. — DOI: 10.1109/JPROC.2021.3058584](#)

22. [Yi Y. High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec / Y. Yi, B. Song // IEEE Signal Processing Letters. — vol. 15, p. 891-894, 2008. — DOI: 10.1109/LSP.2008.2001982](#)

23. [Moroz L. Efficient Floating-Point Division for Digital Signal Processing Application \[Tips & Tricks\] / L. Moroz, V. Samotyv // IEEE Signal Processing Magazine. — vol. 36, no. 1 — P. 159-163 – 2019. — DOI: 10.1109/MSP.2018.2875977](#)

24. [Time scale of system latencies / блог](https://perfsolutions.blogspot.com/2014/05/time-scale-of-system-latencies.html) [Электронный ресурс]. 05.2014. — URL: <https://perfsolutions.blogspot.com/2014/05/time-scale-of-system-latencies.html> (дата обращения: 23.03.2026).

25. [AV1 benchmark / репозиторий](https://github.com/ressitwage/AV1-bench) [Электронный ресурс]. 23.03.2026. — URL: <https://github.com/ressitwage/AV1-bench> (дата обращения: 23.03.2026).

Спасибо за внимание, готов ответить на ваши вопросы

Разработка системы для быстрого декодирования видео в формате AV1

Выполнил студент группы РИС-22-2 Берсенёв Илья
Научный руководитель В. В. Ланин

Бизнес-задача Comexr

- ТАРЕ индекса́тор функционально эквивалентен однонаправленному видеоэнкодеру
- **Проблема:** индексация ТАРЕ в среднем в **24** раза быстрее чем видеodeкодирование
- **Следствие:** узкое место в пайплайне системы
- **Цель:** ускорить декодирование видео чтобы устранить узкое место



Гипотеза, объект и предмет исследования

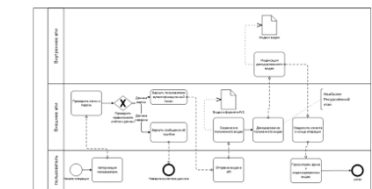
Объект исследования	Предмет исследования
процесс восстановления видеоданных из закодированного состояния	оптимизированная система декодирования видео в формате AV1

Цель исследования: разработать и применить оптимизации видеodeкодера AV1 для выходного разрешения 64 пикселя, спроектировать архитектуру системы декодирования с учетом максимальной производительности

Гипотеза исследования: Алгоритмические оптимизации видеodeкодера и архитектурные оптимизации системы могут повысить совокупную производительность на более чем 30%

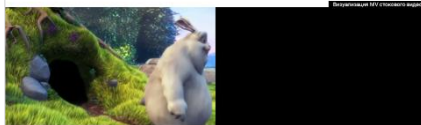
Анализ предметной области

Существующий бизнес процесс



Inter: Motion Vectors

- Motion Vectors (MV) – метод кодирования движущихся частей видео с помощью дельты x и дельты y кадра по сравнению с опорным



Intra предсказание

Intra предсказание представляет из себя вычисление пикселя по пикселю.
H.264: 9 направлений
AV1: 56 направлений, рекурсивное предсказание



Проектирование

Наивная система

- × Не имеет потенциала к масштабированию и оптимизации



Оптимизированная система

- Frontend
 - Wasm, ffmpeg
- Система декодирования
 - Fastapi, ffmpeg, dav1d
- База данных
 - S3(Ceph), postgresql, Redis
- Система маршрутизации
 - Fastapi, websockets
- Система авторизации
 - JWT



Реализация: оптимизации DAV1D



DAV1D: метод оценки

Оборудование:

- CPU: AMD EPYC 9554 (1 ядро)
- OS: Ubuntu 22.04.5, без GUI
- DISK: SSD NVMe

Метрики качества:

- PSNR оригинального и декодированного видео
- Показатель средняя разность luma-компонента

Метрика скорости:

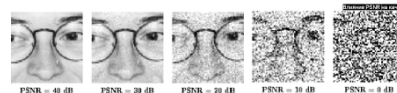
- Скорость от начала декодирования видео до конца декодирования видео
- 3 итерации, Ubuntu 22.04.5 6e3 GUI



DAV1D: тестирование: PSNR

PSNR: вычисление с помощью FFMPEG, numpy

- Минимальное значение: 21.02дБ
- Среднее значение: 26.2дБ
- Максимальное значение: 35.38дБ



Реализация:
оптимизированная система

Оптимизированная система: метод оценки

Оборудование (клиент):

- CPU: Intel i5-12400f, AMD Ryzen H255
- OS: Linux mint 22.3, Windows 10
- DISK: SSD NVMe

Метрики скорости:

- Общая задержка от начала отправки видео до получения уведомления
 - Последовательная отправка видео через скрипт, минимальное время на 5 итерациях
- Общая пропускная способность системы
 - Количество декодированных видео (Мб) за секунду при параллельной отправке видео на декодирование

Тестирование оптимизированной системы

Метрика	Наименовая система	Оптимизированная система
Общая задержка от начала отправки видео на сервер до полного декодирования, в секундах, 5 видео	283,4с	165,9с
Общая пропускная способность системы, в Мб/с	1,98Мб/с	4,28Мб/с

Результаты проекта

Метрика тестирования		Мара	Целевая мера
Скорость декодирования видео (DAVID)		+38,5%	+20%
Общая задержка от начала отправки видео на сервер до полного декодирования, в секундах		-41,4%	-30%
Общая пропускная способность системы, в МБ/с		+116,2%	-
Потеря качества при декодировании, PSNR	Минимум	21,03 дБ	>15дБ
	Среднее	28,2 дБ	
	Максимум	35,38 дБ	
Покладовая метрика потери качества, 0-255	Минимум	2,21	-
	Среднее	6,28	

Код проекта

